ACADEMIC & TECHNICAL ARCHITECTURE REPORT

Automated Chest Radiograph Diagnostic System for Tuberculosis & Pneumonia

A Deep Learning, Real-Time OPD Token Queue & Tele-Radiology Web Ecosystem

Project Title:	TB & Pneumonia AI Medical Screening and Clinic Management System
System Domain:	Medical Imaging, Computer Vision, Telehealth & OPD Workflow Automation
AI Backbone & Training:	DenseNet-121 Trained on 30,000 Balanced Radiographs (Normal, TB, Pneumonia)
Computer Vision:	OpenCV: Monochromatic RGB Variance Filter + Laplacian Blur Detection
Backend Core:	FastAPI (Python 3.9+), Uvicorn ASGI, WebSockets, SQLAlchemy ORM
Frontend Core:	React 18 (Vite), Tailwind CSS, Framer Motion, Axios Interceptors
Database Stack:	PostgreSQL 13/18 (Production) / SQLite3 (Local Dev & Testing)
Security Architecture:	OAuth2 Password Flow, HS256 JWT Bearer Tokens, Bcrypt Password Salts
Academic Origin:	BS Software Engineering Final Year Project, University of Swat

Executive Overview: *This document provides an exhaustive technical specification of the entire project repository. It details every layer of the architecture: how the React frontend interfaces with the FastAPI backend, authentication token lifecycle, model training methodology (30,000 images, Adam, ReduceLROnPlateau, transfer learning on DenseNet-121), computer vision pre-screening filters, relational schema definitions, real-time WebSocket queue synchronization, detailed page-by-page operational analysis, and deployment procedures.*

Table of Contents

1. High-Level System Architecture & Design Philosophy	Section 1
2. Complete Technology Stack Matrix (Frontend, Backend, AI, Database, DevOps)	Section 2
3. How Frontend and Backend Connect (Axios Interceptors, JWT, WebSockets, Multipart, CORS)	Section 3
4. Deep Learning Model Training Pipeline & Computer Vision Quality Sanity Gate	Section 4
5. Relational Database Design, Schema ERD & Cascading Integrity	Section 5
6. File-by-File Repository Breakdown & Architectural Map	Section 6
7. Comprehensive Frontend Page-by-Page Breakdown (Patients, Payments, Predict, TodayTokens, etc.)	Section 7
8. Role-Based Portals & Functional Workflows (Patient, Doctor, Administrator)	Section 8
9. Comprehensive RESTful API & WebSocket Endpoint Directory	Section 9
10. Step-by-Step Transaction Lifecycles (Diagnostic Execution & Real-Time OPD Queue)	Section 10
11. Deployment, Environment Configuration & Operational Troubleshooting Guide	Section 11

Executive Summary & Clinical Rationale

Tuberculosis (TB) and Pneumonia remain two of the leading causes of infectious disease mortality globally. Early radiographic detection via chest X-rays (CXR) is standard clinical practice, yet interpreting these images requires specialized radiologist expertise, which is severely strained in developing regions and overburdened hospitals. Misdiagnosis or delayed diagnosis leads to increased morbidity, transmission, and healthcare costs.

The **TB & Pneumonia AI System** addresses this critical bottleneck by providing an end-to-end medical triage solution. It does not merely serve as an isolated machine learning model; it delivers a comprehensive clinical operating system that unifies **automated radiograph validation**, **instant deep neural classification**, **real-time outpatient department (OPD) queue token orchestration**, and **multi-role collaboration** between patients, doctors, and hospital administrators.

1. High-Level System Architecture & Design Philosophy

The system follows a modern decoupled, multi-tier micro-service-ready architecture. The presentation tier (React SPA) is completely isolated from the application tier (FastAPI), communicating strictly through standardized RESTful JSON payloads, multipart file streams, and bidirectional WebSocket events.

Key Architectural Tenets:

- **Stateless Web Core:** The API server maintains no in-memory user sessions. Authentication is entirely encapsulated in signed JSON Web Tokens (JWT) verified on every incoming request.
- **Fail-Fast Medical Sanity Gate:** Low-quality, corrupted, or non-radiological photos are rejected within 15 milliseconds by OpenCV before invoking heavy neural inference.
- **Real-Time Event Broadcast:** Waiting room displays, doctor call registers, and patient phones synchronize instantaneously via WebSockets without requiring database polling.
- **Role-Based Access Control (RBAC):** Explicit role checks on both frontend router guards and backend FastAPI route dependencies prevent privilege escalation.

2. Complete Technology Stack Matrix

Every dependency in the codebase was deliberately selected to balance high computational speed, medical UI clarity, enterprise security, and long-term maintainability:

Layer	Technology	Version / Library	Role & Clinical Justification
Frontend UI	React	v18.2.0+	Modular component architecture, concurrent UI rendering, hooks-driven state.
Frontend Build	Vite	v5.x+	Native ES module development server, sub-second HMR, optimized Rollup bundling.
Frontend Style	Tailwind CSS	v3.x+	Glassmorphism aesthetic, accessible high-contrast medical palettes, dark/light mode.
Animation	Framer Motion	v11.x+	Hardware-accelerated card transitions, diagnostic progress meters, smooth modals.
Icons	Lucide React	Latest	Pixel-perfect healthcare iconography (stethoscopes, lungs, user badges).
HTTP Client	Axios	v1.6.x+	Configured with custom request/response interceptors for automatic JWT injection.
Client Routing	React Router	v6.x+	Client-side routing, protected route guards, smart dashboard auto-redirects.
Backend Web	FastAPI	v0.104.x+	Asynchronous Python framework, auto OpenAPI/Swagger documentation, DI engine.
ASGI Server	Uvicorn	v0.24.x+	Lightning-fast ASGI production server built on uvloop and httptools.
ORM Engine	SQLAlchemy	v2.0.x+	Python SQL toolkit, database agnostic connection pooling, relational mappings.
Validation	Pydantic	v2.x+	Strict runtime payload schema validation, serialization, and typing enforcement.
Password Hash	Bcrypt / Passlib	Latest	Salted cryptographic hashing of passwords using Blowfish key derivation.
JWT Tokens	Python-Jose	Latest	Cryptographically signs and decodes HS256 JWT tokens for stateless auth.
Deep Learning	PyTorch	v2.1.x+	Dynamic computation graph framework executing DenseNet-121 forward pass.
Vision Models	Torchvision	v0.16.x+	Pretrained model definitions, standard radiographic image tensor transformations.
Computer Vision	OpenCV (cv2)	v4.8.x+	Monochromatic color differential check, Laplacian variance blur detection.
Primary Database	PostgreSQL	v13 / v18	Enterprise ACID relational database with robust indexing and foreign keys.
Containers	Docker Compose	v3.8 Compose	Multi-container orchestration packaging DB, FastAPI backend, and Nginx frontend.

3. How Frontend and Backend Connect (In-Depth Technical Mechanics)

The interaction between React and FastAPI is engineered for zero-latency, high reliability, and airtight session security. The integration is governed by five primary architectural pillars:

3.1 Dynamic API Endpoint Resolution

In frontend/src/services/api.js, the client resolves the backend URL via environment variables. When running locally, Vite falls back to http://localhost:8000/api. In Docker or cloud production, VITE_API_URL points to the production domain.

3.2 Axios Interceptors & JWT Authentication Protocol

Communication security relies on JSON Web Tokens. The login transaction flows as follows:

1. User submits identifier (Phone or CNIC) + Password to /api/auth/login.
2. Server verifies salted Bcrypt hash. Upon success, FastAPI issues an HS256-signed JWT containing { 'sub': phone, 'role': user.role, 'id': user.id, 'exp': timestamp }.
3. React stores this token in localStorage.getItem('token').
4. **Axios Request Interceptor:** Every outgoing HTTP call intercepts the configuration and injects: config.headers.Authorization = 'Bearer ' + token.
5. **Axios Response Interceptor:** If any API call returns HTTP 401 Unauthorized (e.g. token expired), the interceptor automatically purges localStorage and redirects the browser to /login without throwing uncaught UI crashes.

```
// 1. Outgoing Request Interceptor: Attach Bearer JWT Token
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) { config.headers.Authorization = `Bearer ${token}`; }
  return config;
});

// 2. Incoming Response Interceptor: Handle Token Expiration Globally
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token'); localStorage.removeItem('user');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
```

3.3 Binary Multipart/Form-Data File Streaming

For medical image screening (/api/predict) and doctor avatar updates (/api/auth/update-profile), standard JSON payloads are bypassed. The client constructs a FormData object appending the binary image stream. Axios sets the header to multipart/form-data. FastAPI asynchronously streams the raw bytes via UploadFile = File(...), writing them to disk without memory overflow.

3.4 Real-Time WebSockets Engine (/api/ws/queue)

To synchronize patient token calls without polling, a WebSocket channel is maintained by ConnectionManager in backend/app/api/websockets.py. Connected client screens receive events in ~10ms and trigger an automated re-fetch of the queue state.

3.5 CORS & Static File Headers

Cross-Origin Resource Sharing is enabled for http://localhost:5173 via FastAPI's CORSMiddleware. Static image directories (/uploads) are mounted using a custom CORSStaticFiles subclass that forcibly appends Access-Control-Allow-Origin: *, allowing frontend canvas rendering, medical image cropping, and report printing without browser security blocks.

4. Deep Learning Model Training Pipeline & Quality Sanity Gate

This section details the formal training methodology, dataset distribution, loss functions, optimization strategies, and hyperparameter configurations utilized to train the core **DenseNet-121** diagnostic model.

4.1 Dataset Curation, Balancing & Experimental Splitting

To prevent algorithmic bias toward any dominant class, an aggregated repository of **30,000 digital chest radiographs** was systematically curated and perfectly balanced across three diagnostic categories: **10,000 Normal**, **10,000 Tuberculosis (TB)**, and **10,000 Pneumonia** radiographs.

Experimental Partition	Split Ratio (%)	Samples per Class	Total Partition Volume	Role in Optimization
Training Set	70%	7,000 per class	**21,000 images**	Backpropagation gradient descent & weight parameter updates.
Validation Set	15%	1,500 per class	**4,500 images**	Hyperparameter tuning, LR scheduling, and early stopping checkpointing.
Testing Set	15%	1,500 per class	**4,500 images**	Unseen, hold-out final diagnostic metric evaluation.
Total Repository	100%	10,000 per class	**30,000 images**	Fully balanced benchmark dataset.

4.2 Image Preprocessing & Data Augmentation Pipeline

- **CLAHE Contrast Enhancement:** Contrast Limited Adaptive Histogram Equalization (CLAHE) was applied to enhance local radiographic contrast across soft lung tissues, accentuating subtle pulmonary opacities.
- **Dynamic Data Augmentations:** Random horizontal flips (p=0.5), random affine rotations (+/- 10 degrees), subtle scaling (0.9 to 1.1), and shear transforms were applied dynamically per batch to prevent memorization and simulate varying radiological acquisition angles.
- **Spatial Resizing & Normalization:** All images were resized to 224x224 pixels and normalized across RGB channels using ImageNet distribution parameters: Mean = [0.485, 0.456, 0.406] and Std = [0.229, 0.224, 0.225].

4.3 Model Architecture & Transfer Learning Strategy

DenseNet-121 was selected over VGG16 (138 million parameters) and ResNet-50 (25 million parameters). DenseNet-121's compact footprint (~8 million parameters, ~28 MB) allows direct feature reuse across dense blocks while enabling rapid real-time inference on hospital CPU workstations without requiring expensive dedicated GPUs. A custom classification head was attached: Global Average Pooling (GAP) produces a 1024-dimensional feature vector, followed by a **Dropout layer (rate = 0.5)**, and a final fully connected layer (nn.Linear(1024, 3)) yielding class probabilities via **Softmax**.

4.4 Optimization Strategy & Hyperparameter Configuration

Hyperparameter	Configured Strategy / Value	Optimization Purpose & Mechanics
Initial Learning Rate (α)	1×10^{-4} (0.0001)	Prevents destructive updates to pre-trained ImageNet backbone weights.
Optimization Algorithm	Adam (Adaptive Moment Estimation)	Accelerates convergence using adaptive first and second momentum gradients.
Loss Function Objective	Categorical Cross-Entropy Loss (L_{CCE})	Quantifies multi-class divergence: $L = -\sum y_i \log(\blacksquare_i)$.
Mini-Batch Size	32 samples	Balances gradient estimation stability with GPU memory constraints.
Maximum Epochs	50 Epochs	Upper bound on full training iterations.
Learning Rate Scheduler	ReduceLROnPlateau (Factor=0.5, Patience=3)	Halves learning rate when validation loss plateaus for 3 epochs.
Early Stopping Callback	Patience = 5 Epochs (Validation Loss)	Halts training when validation loss ceases improving; restores best checkpoint.
Dropout Regularization	Rate = 0.5	Mitigates co-adaptation between neurons in the final dense layer.

4.5 Stage 1 Pre-Inference Quality Sanity Gate (OpenCV)

Before an uploaded image touches the trained neural network, OpenCV executes two validation checks in under 15ms: (1) **Monochromatic Check:** Rejects non-radiological photos or selfies where channel variance (|R-G| + |G-B| + |R-B|) / 3.0 > 50.0; (2) **Blur Check:** Rejects out-of-focus scans where cv2.Laplacian(gray, cv2.CV_64F).var() < 50.0.

5. Relational Database Design & Schema ERD

The relational schema is defined declaratively using SQLAlchemy ORM (backend/app/db/models.py). The database models establish full referential integrity, foreign key indexes, and bidirectional ORM relationships:

Model / Table	Primary / Foreign Keys	Field Attributes & Types	Relational Role & Business Logic
User (users)	PK: id Indexes: phone, cnic	name, phone (UQ), cnic (UQ), hashed_password, role, profile_pic, specialization, clinic_name, address, gender, age, token_mode, max_tokens, about, timetable, consultation_fee, used_access_code, created_at	Represents all human actors: Patients, Doctors, and Administrators. Stores demographic data, doctor credentials, and practice parameters.
Prediction (predictions)	PK: id FK: user_id -> users.id FK: doctor_id -> users.id	image_path, result ('Normal'|'TB'|'Pneumonia'), confidence (Float), all_scores (JSON string), created_at	Stores every AI diagnostic event. Maintains dual ownership: patient owner and treating doctor. Preserves full probability distribution.
Appointment (appointments)	PK: id FK: doctor_id -> users.id FK: patient_id -> users.id	booking_date, token_number, status ('pending'|'serving'|'completed'), fee_charged, patient_name_override, created_at	Manages clinic queue slots. Locks doctor's fee at time of booking. Supports walk-in patients via patient_name_override.
DoctorPatient (doctor_patients)	PK: id FK: doctor_id -> users.id FK: patient_id -> users.id	Association table for many-to-many relationship.	Links registered patients to their treating doctors, enabling patient discovery and longitudinal tracking.
RegistrationRequest (registration_requests)	PK: id FK: doctor_id -> users.id	name, phone, cnic, gender, age, booking_date, token_number, status, rejection_reason, created_at	Facilitates Guest Booking for unregistered patients. Holds unverified token requests until doctor approval.
SiteSettings (site_settings)	PK: id Index: setting_key (UQ)	setting_key, setting_value	Key-value CMS store allowing administrators to dynamically alter contact numbers, addresses, emails, and hero banners.
ContactMessage (contact_messages)	PK: id	name, email, subject, message, is_read (0|1), created_at	Persists public inquiries submitted through the Contact Us form, accessible via the Admin inbox.

Cascading Integrity & Cleanup Protocols:

To prevent database corruption and orphaned records, account deletion via /api/admin/users/{id} executes an atomic transaction that: (1) deletes all prediction records and unlinks raw image files from the uploads/ directory, (2) purges all appointments where the user is patient or doctor, (3) removes registration requests and doctor-patient associations, and (4) deletes the user entity.

6. File-by-File Repository Breakdown & Architectural Map

Below is the complete architectural mapping of source code files across the backend and frontend codebases:

Path in Codebase	Primary Role & Internal Mechanics
backend/run.py	Entry point for ASGI server; initializes Uvicorn on host 0.0.0.0:8000 with auto-reload.
backend/app/main.py	FastAPI app instantiation, CORS setup, security headers middleware, static file mounting with CORSStaticFiles.
backend/app/core/config.py	Pydantic BaseSettings loading .env: DATABASE_URL, SECRET_KEY, REGISTRATION_KEYS, JWT expiry.
backend/app/db/database.py	SQLAlchemy engine initialization, SessionLocal sessionmaker, and get_db() dependency generator.
backend/app/db/models.py	Declarative SQLAlchemy ORM models: User, Prediction, Appointment, DoctorPatient, RegistrationRequest, SiteSettings, ContactMessage.
backend/app/api/auth_utils.py	Cryptographic utilities: Bcrypt hashing/verification, create_access_token(), and get_current_user_from_token().
backend/app/api/routes.py	Primary API router containing over 30 endpoints for authentication, AI predictions, doctor dashboard, and appointments.
backend/app/api/admin.py	Administrative router: system KPIs, user CRUD with cascading deletions, site settings CMS, contact message management.
backend/app/api/websockets.py	ConnectionManager broadcasting real-time queue update events over WebSocket connections.
backend/app/services/predict.py	PyTorch DenseNet-121 inference service: loads chest_xray_densenet121.pth, applies transforms, and returns Softmax scores.
frontend/src/App.jsx	React root router, route definitions, SmartHome auto-redirector, and ProtectedRoute RBAC wrapper.
frontend/src/services/api.js	Axios client configuration, Bearer token injection interceptor, global 401 redirect interceptor, API helper methods.
frontend/src/services/AuthContext.jsx	React Context provider managing user state, login(), logout(), updateUser(), and token persistence.
frontend/src/pages/Patients.jsx	Clinical patient directory: badge grid, live CNIC/Phone search, walk-in register modal, link modal, and full medical dossier.
frontend/src/pages/Payments.jsx	Doctor financial analytics: collected consultation fees, completed appointment tallies, and revenue breakdowns.
frontend/src/pages/Predict.jsx	AI diagnostic screening UI: file dropzone, live patient search, today's queue selector, and ResultCard render.
frontend/src/pages/TodayTokens.jsx	Real-time OPD queue manager: next patient caller, FeeModal (full/half/free), walk-in token creator, and WebSocket sync.
frontend/src/pages/DoctorDashboard.jsx	Doctor cockpit: linked patients list, Recharts area/pie disease distribution charts, appointment metrics, and practice summary.
frontend/src/pages/History.jsx	Diagnostic archives: search filters, full image modal viewer, condition badges, and client-side A4 PDF medical report export.
frontend/src/pages/BookAppointment.jsx	Patient booking interface: doctor directory browser, calendar date selection, and visual token slot selector.
frontend/src/pages/GuestBooking.jsx	Unregistered patient portal to request appointments and check approval status by CNIC/phone.
frontend/src/pages/AdminDashboard.jsx	Admin portal: platform stats, user management (role change/delete), contact messages inbox, and dynamic CMS settings.
frontend/src/pages/Profile.jsx	Account and clinic settings: Monday-Sunday weekly timetable editor, consultation fee, token mode, and profile avatar.
frontend/src/components/UploadBox.jsx	Drag-and-drop file upload component with instant image thumbnail preview and client-side format validation.
frontend/src/components/ResultCard.jsx	Diagnostic result card displaying prediction badge, animated confidence bars, clinical advice, and print/export buttons.

7. Comprehensive Frontend Page-by-Page Architecture & Operation

This section provides an exhaustive analysis of every user interface page in the system, detailing its clinical purpose, interactive components, client-side validation rules, and backend communication endpoints.

7.1 Patients.jsx — The Clinical Patient Directory & Dossier

File Path: frontend/src/pages/Patients.jsx (53.3 KB) | Role: Doctor & Admin

Patients.jsx serves as the doctor's comprehensive patient management cockpit. It displays all patients linked to the doctor in an animated, responsive badge matrix. Key functionalities include:

- **Live Multi-Field Search & Date Filter:** Instant filtering across Name, 13-digit CNIC, and Phone number. Includes a date-tracker card toggling between 'Today\'s Load' and 'All-Time Registry'.
- **Walk-in Patient Registration Modal (handleRegister):** Enforces strict identity standards: CNIC must be exactly 13 digits (/^\d{13}$/) and Phone must be an 11-digit mobile starting with '03' (/^03\d{9}$/). If the patient already exists in the database, it automatically catches the 400 error and switches directly into 'Linking Mode'.
- **'Link Existing Patient' Modal (handleLink):** Searches the hospital directory by CNIC/Phone and creates a record in doctor_patients, associating existing accounts with the doctor.
- **Patient Medical Dossier Modal:** Clicking any patient badge opens a full overlay detailing demographics, CNIC, phone, registration date, and a complete **Clinical Screening History** timeline. Each past scan displays an X-ray thumbnail, date, condition badge (Normal / TB / Pneumonia), and percentage score. Direct action buttons allow the doctor to immediately **'Start AI Screening'** (pre-filling Predict.jsx) or **'Book Token'**.
- **Card Hover Quick-Actions:** Direct buttons on each tile to Edit patient profile (calls PUT /api/doctor/update-patient/{id}), Remove/Unlink patient (calls DELETE /api/doctor/unlink-patient/{id}), or Book Token.

7.2 Payments.jsx — Clinic Financials & Fee Tracking

File Path: frontend/src/pages/Payments.jsx (10.8 KB) | Role: Doctor & Admin

Payments.jsx tracks doctor revenues and consultation fee collections. It calculates earnings exclusively from completed appointments:

- **Financial KPI Cards:** Three real-time summary cards: *All-Time Revenue* (cumulative PKR collected), *This Month\'s Revenue* (current calendar month total), and *Today\'s Earnings*.
- **Grouped Transaction History Table:** Chronological ledger grouping consultations by date with daily totals (e.g. ■ Today (Daily Total: Rs. 14,000)). Displays patient name, phone, token number, completion badge, and exact fee collected.
- **Filtering & Search:** Live date-picker filter and patient name search with instant 'Clear Filters' capability.
- **API Integration:** Queries GET /api/doctor/payments with JWT Bearer authentication.

7.3 Predict.jsx — AI Diagnostic Screening Workspace

File Path: frontend/src/pages/Predict.jsx (11.5 KB) | Role: Patient, Doctor, Admin

The core diagnostic center where chest X-rays are uploaded and evaluated by the neural network:

- **Dual Mode Setup:** For patients, it provides a simple drag-and-drop dropzone. For doctors, it features a live patient identity verification search bar and a dropdown selector populated with today\'s scheduled OPD patients.
- **Computer Vision Quality Sanity Alerts:** Catches server-side OpenCV 400 responses: alerts the user if an uploaded image is a colored photo/selfie (non-monochromatic) or if the image is too blurry/out-of-focus (Laplacian variance < 50).
- **ResultCard.jsx Presentation:** Renders the primary condition banner, animated probability bars for Normal, TB, and Pneumonia, clinical triage guidance, and full-resolution print capabilities.

7.4 TodayTokens.jsx — Live OPD Queue & Waiting Room Cockpit

File Path: frontend/src/pages/TodayTokens.jsx (23.5 KB) | Role: Doctor & Admin

Manages active clinic sessions and real-time waiting room token flow:

- **Queue Status Progression:** Manages appointments transitioning through pending → serving → completed.
- **FeeModal Component:** When a doctor marks an appointment complete, an interactive popup presents presets: *Full Fee* (standard rate), *Half Fee* (discounted follow-up), *Free (Rs. 0)* (waived fee), or *Custom Fee*. The confirmed fee is persisted to the database.
- **Shift Controls:** Includes *'Move to Tomorrow'* (reschedules an individual patient with the next available token) and *'End Shift'* (moves all remaining pending patients to tomorrow in bulk).
- **Print Token Slip:** Generates a formatted physical token receipt containing doctor name, clinic address, patient details, date, and token number.
- **Real-Time Synchronization:** Combines a 15-second background poll with a live WebSocket listener (/api/ws/queue) to push status updates instantly to waiting room monitors.

7.5 DoctorDashboard.jsx — Doctor Operations Command Center

File Path: frontend/src/pages/DoctorDashboard.jsx (60.6 KB) | Role: Doctor & Admin

The primary landing dashboard for authenticated medical practitioners:

- **Clinical Analytics & Recharts Visualization:** Renders Area Charts tracking patient inflow trends and Pie Charts displaying the diagnostic breakdown of Normal vs TB vs Pneumonia cases.
- **Active Queue Monitor:** Displays the currently serving token and the upcoming queue directly on the dashboard home screen.
- **Embedded Diagnostic Quick-Scan:** Allows doctors to analyze an X-ray on the spot without leaving the dashboard.
- **Recent Screenings Feed:** Chronological list of latest diagnoses with image thumbnails and instant report links.

7.6 History.jsx — Diagnostic Archives & PDF Report Generator

File Path: frontend/src/pages/History.jsx (58.3 KB) | Role: Patient, Doctor, Admin

Comprehensive screening history and clinical documentation generator:

- **Role-Adaptive View:** Patients view their personal chronological scans; doctors view scans across all their registered patients, grouped neatly by patient profile.
- **Full-Resolution Radiograph Viewer:** Modal inspection tool allowing doctors to view the original uncompressed radiograph with contrast enhancement to detect subtle lung infiltrates.
- **Client-Side A4 Medical Report Generation (downloadPDF):** Uses html2pdf.js with CORS-injected image headers to compile an official hospital diagnostic report. Formatted strictly for A4 print dimensions with hospital header, patient demographics, AI confidence meters, and physician signature space.
- **Record Management:** Doctors can permanently delete erroneous scans, triggering cascading database deletion and disk image cleanup.

7.7 BookAppointment.jsx & GuestBooking.jsx — OPD Appointment Engines

File Paths: BookAppointment.jsx (68.6 KB) & GuestBooking.jsx (26.9 KB)

- **BookAppointment.jsx (Registered Users):** Patients select a doctor from the clinical directory, pick an upcoming date, and view an interactive visual token matrix. Booked slots appear locked; available slots can be reserved with one click. Locks consultation fee at booking time.
- **GuestBooking.jsx (Public / Unregistered):** Allows first-time walk-in patients without an account to request an appointment slot. Submits a registration request to the doctor's queue (registration_requests table) and provides a live status inquiry tool checking approval by CNIC or Phone.

7.8 AdminDashboard.jsx & Profile.jsx — Management & Configuration

File Paths: AdminDashboard.jsx (41.6 KB) & Profile.jsx (33.6 KB)

- **AdminDashboard.jsx:** Platform governance portal. Features system KPI counters, full user directory management (role elevation/demotion, cascading account deletion), dynamic CMS site settings editor (contact numbers, addresses, hero banners), and contact form message inbox.
- **Profile.jsx:** Personal and clinic practice settings. Doctors configure their standard consultation fee, token mode (Fixed daily limit vs Dynamic), weekly timetable (Monday-Sunday opening/closing hours), specialization, clinic address, and biography.

7.9 Authentication & Informational Pages

- **Login.jsx (5.0 KB):** Dual-identifier authentication supporting either Phone number or CNIC paired with password.
- **Register.jsx (9.8 KB):** Dual-role registration. Doctor signups require authorized medical access codes validated against REGISTRATION_KEYS.
- **ForgotPassword.jsx (5.6 KB):** Identity verification matching Phone + CNIC before permitting password resets.
- **Home.jsx, About.jsx, Contact.jsx, Terms.jsx, Privacy.jsx:** Responsive public landing pages with disease epidemiology, medical disclaimers, and interactive contact forms.

8. Role-Based Portals & Functional Workflows

The system enforces strict role-based separation of concerns across three user classes:

8.1 Patient Experience & Workflow

- **Dual-Identity Registration:** Patients register with Name, Phone, and National ID (CNIC). Login accepts either Phone or CNIC paired with their password.
- **Self-Screening Upload:** Patients upload their CXR scan in /predict. The AI renders instant triage results with visual confidence meters and recommends whether to consult a pulmonologist.
- **Screening Archive:** In /history, patients can review past diagnostic scans, filter by date or result (Normal, TB, Pneumonia), and view high-resolution image previews.
- **Appointment Booking:** In /book, patients browse doctors, select dates, view booked vs free tokens, and instantly lock a slot. Token numbers are issued consecutively.

8.2 Doctor Workspace & Clinical Workflow

- **Authorized Registration:** Doctors can only register by supplying a valid access code matching the comma-separated REGISTRATION_KEYS environment variable.
- **In-Clinic Diagnostic Workspace:** Doctors screen X-rays on behalf of patients. An intelligent live search field verifies patients by CNIC or Phone in real time, automatically associating the scan with that patient's permanent record.
- **Today's Live OPD Queue:** In /today-tokens, doctors manage their active clinic queue. Doctors can advance tokens to **Serving**, mark them **Completed**, check in patients, or move overflow patients to tomorrow's queue.
- **Walk-in & Family Tokens:** Doctors can assign appointments with a patient_name_override to cater to walk-in patients or families sharing a single phone number.
- **Practice & Token Settings:** In /profile, doctors configure their consultation fees, token mode (fixed vs dynamic daily limits), timetable, clinic address, and biography.
- **Financial Tracking:** Accessible via /payments, providing analytics on total collected consultation fees, daily revenues, and completed patient tallies.

8.3 Administrator Management & CMS Operations

- **Executive KPI Statistics:** Total registered users, doctor count, patient count, total AI predictions executed, and total booked appointments.
- **User Directory Management:** Ability to view all users, edit demographic data, promote/demote user roles (Patient ↔ Doctor ↔ Admin), and perform clean deletions.
- **Dynamic Site Content CMS:** Real-time configuration of support emails, hotline numbers, WhatsApp links, physical clinic addresses, and landing page hero copy without code changes.
- **Contact Message Center:** View, mark as read, and delete inquiries submitted through the public Contact page.

9. Comprehensive RESTful API & WebSocket Endpoint Directory

The following matrix documents all endpoints provided by the backend API routers (routes.py and admin.py):

Endpoint & HTTP Method	Authorization	Payload / Parameters	Behavior & Clinical Operation
POST /api/auth/register	Public	JSON: name, phone, cnic, password, role, access_code	Registers user; checks uniqueness; validates doctor access code; hashes password.
POST /api/auth/login	Public	JSON: identifier, password	Authenticates via phone/CNIC; returns signed JWT Bearer token and user profile.
POST /api/auth/update-profile	Authenticated	Multipart Form-Data	Updates user details, clinic address, fees, timetable, and profile photo.
POST /api/auth/reset-password	Public	JSON: phone, cnic, new_password	Verifies phone and CNIC identity match before updating hashed password.
POST /api/predict	Authenticated	Multipart: file (image), patient_identifier	Runs OpenCV sanity checks & DenseNet121 inference; saves record to DB.
GET /api/patient/history	Authenticated	Bearer Token	Fetches chronological screening history with image URLs and confidence scores.
DELETE /api/doctor/delete-prediction/{id}	Doctor Only	Path: prediction_id	Deletes screening entry from DB and unlinks image file from disk storage.
POST /api/doctor/link-prediction-to-patient	Doctor Only	JSON: prediction_id, patient_id	Reassigns an unlinked screening record to a specific patient's profile.
GET /api/doctor/search-patient	Doctor Only	Query: ?identifier=	Searches patient by CNIC or phone; returns existence and demographic data.
POST /api/doctor/register-patient	Doctor Only	Multipart / Form-Data	Creates patient record on-spot during clinic visit and links to doctor.
GET /api/doctor/dashboard	Doctor Only	Bearer Token	Returns summary metrics, today's queue count, and linked patient registry.
GET /api/doctor/payments	Doctor Only	Bearer Token	Computes total earned revenue, completed appointment counts, and daily fees.
GET /api/doctors/list	Public	None	Returns list of registered doctors with specialty, fee, and clinic details.
GET /api/appointments/booked-tokens	Public	Query: ?doctor_id=&date;=	Returns array of allocated token integers for specific doctor and date.
POST /api/appointments/book	Authenticated	JSON: doctor_id, date, token_number	Reserves appointment slot; locks current fee; triggers WebSocket queue push.
GET /api/appointments/doctor-today	Doctor/Admin	Query: ?date=	Returns all scheduled patient tokens for the doctor's active clinic session.
POST /api/appointments/update-status	Doctor/Admin	JSON: appointment_id, status	Updates token status (pending/serving/completed); broadcasts WS update.
POST /api/appointments/move-to-tomorrow	Doctor/Admin	JSON: appointment_id	Reschedules token to next date with auto-allocated sequential token number.
WS /api/ws/queue	Public	Bidirectional WS connection	Live socket delivering queue advancement events to waiting room displays.
GET /api/admin/stats	Admin Only	Bearer Token	Aggregates system-wide analytics: users, doctors, predictions, appointments.
GET /api/admin/users	Admin Only	Bearer Token	Returns full user directory excluding hashed passwords.
POST /api/admin/users/{id}/role	Admin Only	JSON: role ('patient'|'doctor'|'admin')	Promotes or demotes user role; prevents admin self-revocation.
DELETE /api/admin/users/{id}	Admin Only	Path: user_id	Performs cascading deletion across predictions, appointments, and disk files.
GET /api/admin/settings/contact	Public / Admin	None	Fetches dynamic site settings (contact numbers, emails, addresses, banners).
PUT /api/admin/settings/contact	Admin Only	JSON: key-value settings	Updates dynamic CMS settings persisted in site_settings table.

10. Step-by-Step Transaction Lifecycles

To illustrate the runtime execution path, this section traces the two primary clinical operations in the system:

10.1 Transaction 1: AI Diagnostic Screening Execution Trace

1. **Image Selection:** User drops CXR image onto UploadBox.jsx. File is verified client-side for MIME type (image/jpeg, image/png).
2. **Diagnostic Metadata Binding:** If doctor is screening, doctor inputs patient CNIC/phone in Predict.jsx. Asynchronous call to /api/doctor/search-patient confirms patient record.
3. **Multipart HTTP Dispatch:** runAnalysis() constructs FormData, appends file and patient_identifier, and issues POST request to /api/predict with JWT Bearer header.
4. **File Persistence:** FastAPI receives stream, validates extension, generates UUID filename (e.g. f47ac10b-...png), and saves bytes into uploads/.
5. **OpenCV Quality Gate:** OpenCV inspects image. If channel differences > 50.0 (non-grayscale photo) or Laplacian variance < 50.0 (blurry scan), file is unlinked and HTTP 400 exception is returned.
6. **PyTorch Inference:** Image is resized to 224x224, normalized via ImageNet parameters, and passed through DenseNet-121. Softmax produces probability scores: { 'Normal': 0.04, 'Pneumonia': 0.02, 'TB': 0.94 }.
7. **Database Commitment:** Record is inserted into predictions table with result, confidence, all_scores, user_id, and doctor_id.
8. **Visual Presentation:** ResultCard.jsx displays animated confidence bars, clinical recommendations, and options to print or view full-screen.

10.2 Transaction 2: OPD Token Booking & Live Waiting Room Broadcast Trace

1. **Availability Fetch:** Patient navigates to BookAppointment.jsx. Component queries /api/appointments/booked-tokens?doctor_id=X&date;=YYYY-MM-DD.
2. **Interactive Slot Selection:** Booked tokens appear greyed out; available tokens appear clickable. Patient selects Token #12.
3. **Reservation Request:** Client dispatches POST to /api/appointments/book with selected slot and doctor ID.
4. **Atomic Record Locking:** Server checks token conflict. If slot is open, appointment record is created with status = 'pending' and doctor's current fee is locked into fee_charged.
5. **WebSocket Event Broadcast:** Route triggers manager.broadcast({ 'action': 'queue_update', 'doctor_id': X, 'date': 'YYYY-MM-DD' }).
6. **Real-Time Waiting Room Update:** All connected clinic monitors and patient handsets receive payload, immediately re-fetching queue state without user refresh.
7. **Doctor Consultation:** Doctor opens TodayTokens.jsx, clicks 'Serve', advancing status to serving, triggering another WebSocket broadcast.

11. Deployment, Environment Configuration & Troubleshooting

This section provides complete operational guidance for setting up, running, and troubleshooting the platform in local and production environments.

11.1 Environment Variables Reference (.env)

Variable Name	Default Value / Example	Operational Purpose
DATABASE_URL	postgresql://postgres:pass@localhost:5432/tb_pneumonia_db	Database connection string. Supports PostgreSQL and SQLite (e.g. sqlite:///./medical_db.sqlite).
SECRET_KEY	your_super_secret_key_change_in_prod	Cryptographic key used to sign HS256 JWT access tokens.
ALGORITHM	HS256	Symmetric encryption algorithm for JWT encoding and decoding.
ACCESS_TOKEN_EXPIRE_MINUTES	10080 (7 days)	Token lifespan before requiring re-authentication.
REGISTRATION_KEYS	DOC123,PULMO2024,1002,1003	Comma-separated medical access codes authorized for Doctor account creation.
VITE_API_URL (Frontend)	http://localhost:8000	Base URL pointing React client to FastAPI server.

11.2 Step-by-Step Local Execution

Backend API: cd backend && pip install -r requirements.txt && python run.py (Port 8000)
Frontend Client: cd frontend && npm install && npm run dev (Port 5173)
Docker Compose: cd docker && docker-compose up --build -d (Automated Full Cluster)

11.3 Common Operational Issues & Prescribed Solutions

Symptom / Error	Root Cause	Prescribed Solution
CORS Error in Browser Access to XMLHttpRequest blocked	Frontend origin missing from FastAPI CORSMiddleware.	Verify http://localhost:5173 is in allow_origins in backend/app/main.py.
Invalid Access Code on Register	Entered access code does not match REGISTRATION_KEYS.	Check .env in backend or use valid code (e.g. 1002, DOC123).
'Image is too blurry' HTTP 400	Laplacian variance of uploaded image is < 50.0.	Upload a clear, high-resolution, focused chest X-ray image without camera shake.
'Invalid image format' HTTP 400	Color differential check > 50.0 (non-grayscale photo).	Upload an authentic medical radiograph instead of a selfie or color photograph.
Database Connection Refused	PostgreSQL service not running on port 5432.	Start PostgreSQL service or switch DATABASE_URL to SQLite fallback: sqlite:///./medical_db.sqlite.

Engineering Signoff & Concluding Statement: *The TB & Pneumonia AI Medical Screening System is fully assembled, validated, and documented. This comprehensive technical report serves as the complete academic and operational guide for evaluators, software engineers, and clinical stakeholders.*